Proyectos

Pepe Calderón, Fernando Sola, Daniel Ayala, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz

Contenido

1. Requisitos 2
2. Modelo Conceptual 3
3. Diagrama de clases 3
4. Diagrama de objetos 4
5. Modelo Relacional 4
6. Intensión 4
7. Álgebra relacional 5
8. Modelo Tecnológico (MariaDB) 5
9. Script SQL para crear la base de datos 5
10. Script SQL para la carga inicial de datos 7
11. Consultas 8

Versión PDF disponible

Proyectos

1. Requisitos

Catálogo de Requisitos La trascripción que aparece a continuación corresponde a una entrevista a una ingeniera de software que necesita un sistema de información para ayudarle en la gestión de sus proyectos.

- Pregunta: Bien, coménteme cuál sería el principal objetivo del sistema que usted necesita.
- Respuesta: Bueno, básicamente lo que quiero es un gestor de proyectos sencillo, que me permita gestionar las tareas asociadas y asignárselas a los empleados.
- P: Empecemos entonces por los proyectos, ¿qué información quiere que gestione el sistema sobre ellos?
- R: Básicamente, el nombre del proyecto, una descripción y el presupuesto que tiene. Puede que luego sea interesante añadirle más información, pero de momento me valdría con eso.
- P: De acuerdo, ha hablado antes de tareas. ¿Me puede explicar ese concepto?
- R: Sí, claro. Nosotros descomponemos los proyectos en tareas, lo que se conoce como WBS (Work Breakdown Structure). Cada tarea tiene un identificador que usamos para referirnos a ella, p.e. la tarea T-28F. También tienen una descripción y una estimación del coste de su realización. En el caso de que una tarea sea muy compleja, puede descomponerse en subtareas, que también pueden descomponerse recursivamente.
- P: Entiendo, ¿esas tareas tienen algún orden especial?
- R: El que le vayamos dando al crearlas, aunque luego podríamos cambiarlo.
- P: Entiendo, un proyecto tiene una secuencia de tareas que a su vez pueden tener una secuencia de subtareas y así recursivamente. Debemos conocer el orden. ¿Qué más hay que saber sobre las tareas?
- R: Pues que una vez que creamos la WBS, vamos asignando las tareas a los empleados.
- P: ¿En qué consiste una asignación de una tarea a un empleado?
- R: Básicamente, se encarga a un empleado la realización de una tarea con una fecha de inicio y otra de fin.
- P: ¿A cuántos empleados se le asigna una misma tarea?
- R: Cuando decidimos a quién asignársela, a uno sólo. Nuestras tareas simples están pensadas para que las pueda realizar un solo empleado.
- P: ¿Qué es una tarea simple?
- R: Una tarea que no está descompuesta en tareas más simples.
- P: Entiendo, sólo se asignan tareas simples, ¿no es así?
- R: Bueno, no necesariamente. Podemos asignar tareas complejas para su supervisión.
- P: Entiendo, se puede asignar cualquier tarea (compleja o simple) a cualquier empleado.
- R: Sí, aunque eso depende del rol que tenga el empleado en el proyecto.
- P: Explíqueme eso.
- R: Bueno, en cada proyecto se definen una serie de roles y cada empleado puede desempeñar distintos roles en distintos proyectos. Por ejemplo, uno puede ser el director de un proyecto y a la vez ser el responsable de pruebas de otro proyecto.
- P: Entiendo, ¿un mismo empleado puede jugar más de un rol en un mismo proyecto?

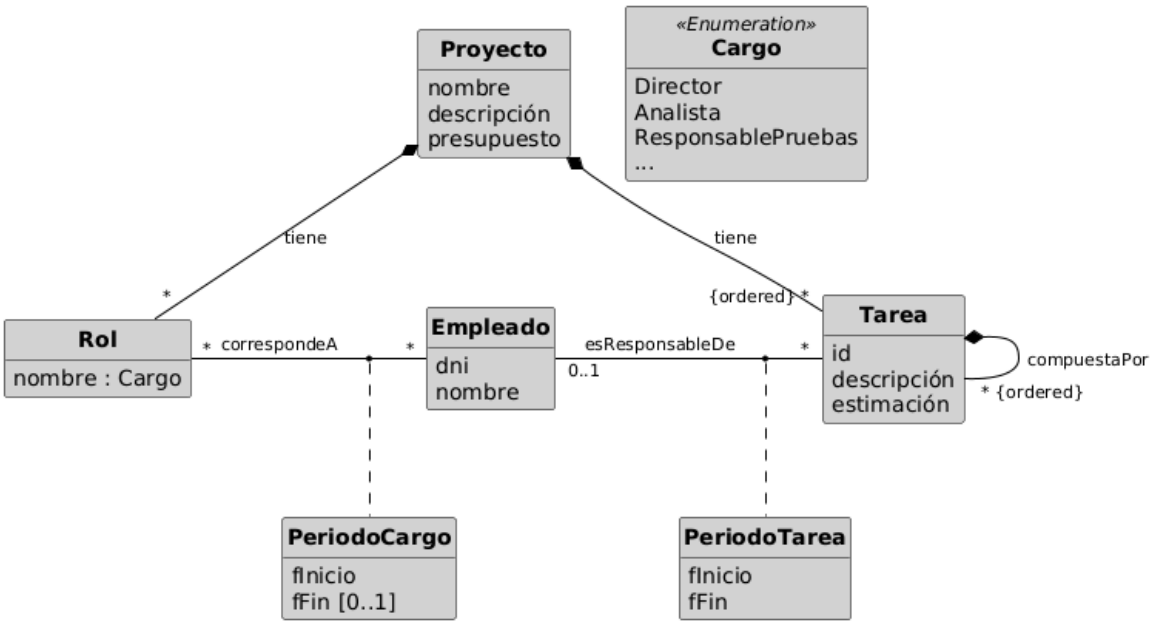
- R: No es frecuente, pero podría ocurrir, especialmente en proyectos pequeños.
- P: Una vez establecido un rol de un empleado en un proyecto, ¿puede cambiar?
- R: Sí, reasignamos los empleados según los proyectos que van saliendo.
- P: Entonces, le interesa saber desde qué fecha hasta qué fecha un empleado ha desempeñado un rol en un proyecto ¿no?
- R: Sí, quiero saber qué empleados han ido pasando por cada proyecto y en qué fechas.
- P: Entiendo que cada rol está asociado a un cargo predefinido, ¿es así?
- R: Sí, claro. Los nombres de los roles coinciden con los cargos habituales: director, analista, responsable de pruebas, etc. Tenemos definidos los roles según la norma ISO-9001.
- P: Muy bien, creo que con eso tengo para una primera versión. Gracias.

2. Modelo Conceptual

Modelo conceptual

3. Diagrama de clases

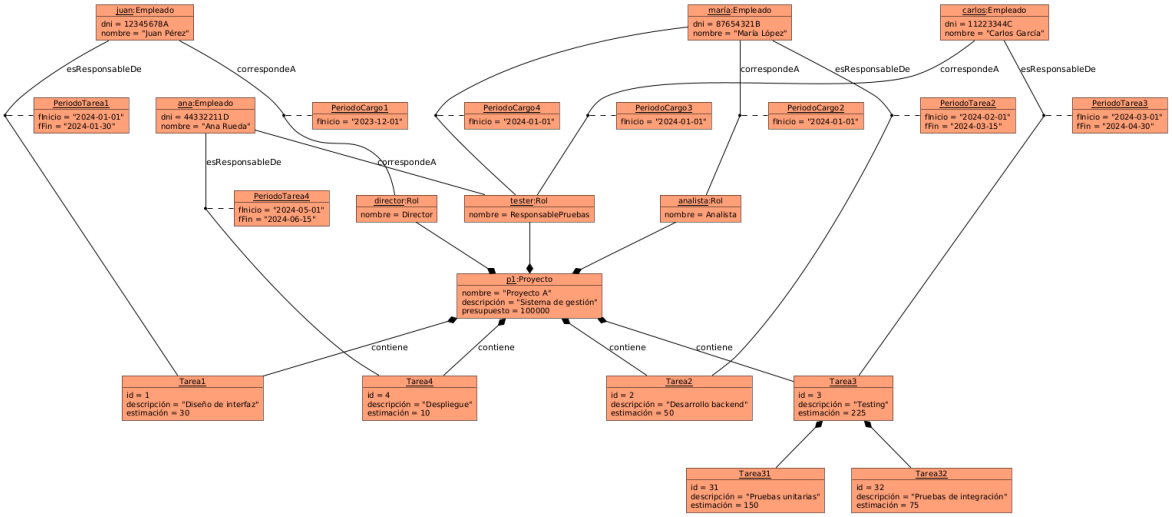
- **Proyecto**: compone un conjunto ordenado de **Tarea** y también "posee" los **Rol** del proyecto, con un ciclo de vida ligado al proyecto.
- **Tarea**: se descompone en subtareas ordenadas mediante auto-composición, formando una estructura jerárquica.
- **Relación Tarea–Empleado**: cada **Tarea** tiene como mucho un **Empleado** responsable (0..1), mientras que un empleado puede llevar varias tareas. **PeriodoTarea** modela el intervalo temporal de esa responsabilidad.
- **Relación Empleado–Rol**: **Empleado** y **Rol** se relacionan en una cardinalidad N:M.
- **PeriodoCargo** añade las fechas de vigencia del cargo (con `fFin` opcional).
- El tipo de rol se restringe mediante el enumerado **Cargo** (Director, Analista, ResponsablePruebas, etc.).



4. Diagrama de objetos

- **p1:Proyecto** contiene **Tarea1**–**Tarea4**. **Tarea3** se descompone en **Tarea31** y **Tarea32**, ilustrando la jerarquía.
- **Asignación de responsables**:
  - **juan** → **Tarea1**
  - **maría** → **Tarea2**
  - **carlos** → **Tarea3**
  - **ana** → **Tarea4**

  Cada par (**Tarea**, **Empleado**) tiene su **PeriodoTarea** con fechas de inicio y fin.
- Los empleados están vinculados a roles (por ejemplo, **juan**–**director**, **maría**–**analista**).
- Se observa que un empleado puede ejercer varios roles en el tiempo (por ejemplo, **maría** puede asumir también **tester**) y que cada tarea tiene un único responsable vigente, respetando las cardinalidades.



5. Modelo Relacional

Modelo relacional

6. Intensión

```
Proyectos(proyectoId, nombre, presupuesto)
  PK(proyectoId)
Roles(rolId, proyectoId, nombre)
  PK(rolId)
  FK(proyectoId) / Proyectos
Tareas(tareaId, proyectoId, orden, id, descripcion, estimacion)
  PK(tareaId)
  FK(proyectoId) / Proyectos
  AK(proyectoId, orden)
Subtareas(subtareaId, tareaId, orden)
  PK(subtareaId, tareaId)
  FK(tareaId) / Tareas
  AK(subtareaId, orden)
Empleados(empleadoId, dni, nombre)
  PK(empleadoId)
  AK(dni)
PeriodosCargos(periodoCargoId, empleadoId, rolId, fInicio, fFin)
```

```
  PK(periodoCargoId)
  FK(empleadoId) / Empleados
  FK(rolId) / Roles
  AK(empleadoId, rolId)
PeriodosTareas(periodoTareaId, empleadoId, tareaId, fInicio, fFin)
  PK(periodoTareaId)
  FK(empleadoId) / Empleados
  FK(tareaId) / Tareas
  AK(empleadoId, tareaId)
```

7. Álgebra relacional

- Empleados con roles en proyectos:

$$ER \leftarrow Empleados \bowtie PeriodosCargos \bowtie Roles$$

- Empleados con tareas en proyectos:

$$ET \leftarrow Empleados \bowtie PeriodosTareas \bowtie Tareas$$

- Empleados que trabajan en proyectos (roles o tareas):

$$EP \leftarrow ER \cup ET$$

- Empleados que trabajan en proyectos (roles y tareas):

$$EP \leftarrow ER \cap ET$$

- Tareas asignadas a empleados en el proyecto 1:

$$TareasEmpleadoP1 \leftarrow \underset{proyectoId=1}{\sigma}(ET)$$

- Número de tareas por empleado (proyecto 1):

$$NumTareasEmpleadoP1 \leftarrow \overset{\text{COUNT(*)}}{\gamma} empleadoId(TareasEmpleadoP1)$$

- Listado de roles por proyecto:

$$ProyectosRoles \leftarrow \gamma empleadoId(ER)$$

- Listado de tareas por proyecto:

$$ProyectosTareas \leftarrow \gamma empleadoId(ET)$$

- Número de subtareas por tarea:

$$Subtareas \leftarrow \overset{\text{COUNT(*)}}{\gamma} tareaId(Tareas \bowtie Subtareas)$$

- Empleados con subtareas:

$$EST \leftarrow ET \bowtie Subtareas$$

8. Modelo Tecnológico (MariaDB)

Modelo tecnológico (MariaDB)

9. Script SQL para crear la base de datos

Proyectos/createDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2024
-- Descripción: script para crear el esquema de la BD
--

-- Crear la base de datos
DROP DATABASE IF EXISTS ProyectosDB;
CREATE DATABASE ProyectosDB;
USE ProyectosDB;

-- Crear la tabla Proyectos
CREATE OR REPLACE TABLE Proyectos (
    proyectoId INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(255) NOT NULL,
    presupuesto DECIMAL(15, 2) NOT NULL
);

-- Crear la tabla Roles
CREATE OR REPLACE TABLE Roles (
    rolId INT AUTO_INCREMENT PRIMARY KEY,
    proyectoId INT,
    nombre VARCHAR(255) NOT NULL,
    FOREIGN KEY (proyectoId) REFERENCES Proyectos(proyectoId)
);

-- Crear la tabla Tareas
CREATE OR REPLACE TABLE Tareas (
    tareaId INT AUTO_INCREMENT PRIMARY KEY,
    proyectoId INT,
    orden INT NOT NULL,
    id CHAR(12) NOT NULL,
    descripcion TEXT NOT NULL,
    estimacion INT NOT NULL,
    FOREIGN KEY (proyectoId) REFERENCES Proyectos(proyectoId),
    UNIQUE (tareaId, orden)
);

-- Crear la tabla Subtareas
CREATE OR REPLACE TABLE Subtareas (
    subtareaId INT AUTO_INCREMENT,
    tareaId INT,
    orden INT NOT NULL,
    PRIMARY KEY (subtareaId, tareaId),
    FOREIGN KEY (tareaId) REFERENCES Tareas(tareaId),
    UNIQUE (subtareaId, orden)
);

-- Crear la tabla Empleados
CREATE OR REPLACE TABLE Empleados (
    empleadoId INT AUTO_INCREMENT PRIMARY KEY,
    dni VARCHAR(20) NOT NULL UNIQUE,
    nombre VARCHAR(255) NOT NULL
);

-- Crear la tabla PeriodosCargos
```

```sql
CREATE OR REPLACE TABLE PeriodosCargos (
    periodoCargoId INT AUTO_INCREMENT PRIMARY KEY,
    empleadoId INT,
    rolId INT,
    fInicio DATE NOT NULL,
    fFin DATE,
    FOREIGN KEY (empleadoId) REFERENCES Empleados(empleadoId),
    FOREIGN KEY (rolId) REFERENCES Roles(rolId),
    UNIQUE (empleadoId, rolId)
);

-- Crear la tabla PeriodosTareas
CREATE OR REPLACE TABLE PeriodosTareas (
    periodoTareaId INT AUTO_INCREMENT PRIMARY KEY,
    empleadoId INT,
    tareaId INT,
    fInicio DATE NOT NULL,
    fFin DATE,
    FOREIGN KEY (empleadoId) REFERENCES Empleados(empleadoId),
    FOREIGN KEY (tareaId) REFERENCES Tareas(tareaId),
    UNIQUE (empleadoId, tareaId)
);
```

10. Script SQL para la carga inicial de datos

Proyectos/populateDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Script para cargar los datos de pruebas
--

-- Insertar datos de ejemplo en la tabla Proyectos
INSERT INTO Proyectos (proyectoId, nombre, presupuesto) VALUES
    (1,'Proyecto A', 100000)
;

-- Insertar datos de ejemplo en la tabla Roles
INSERT INTO Roles (rolId, proyectoId, nombre) VALUES
    (1, 1, 'Director'),
    (2, 1, 'Analista'),
    (3, 1, 'Responsable de pruebas')
;

-- Insertar datos de ejemplo en la tabla Tareas
INSERT INTO Tareas (tareaId, proyectoId, orden, id, descripcion, estimacion)
VALUES
    (1, 1, 1, 'T-DI', 'Diseño de interfaz', 30),
    (2, 1, 2, 'T-DB', 'Desarrollo de BackEnd', 50),
    (3, 1, 3, 'T-T', 'Testing', 225),
    (4, 1, 4, 'T-TPU', 'Pruebas unitarias', 150),
    (5, 1, 5, 'T-TPI', 'Pruebas de integración', 75),
    (6, 1, 6, 'T-D', 'Despliegue', 10)
;
```

```sql
-- Insertar datos de ejemplo en la tabla Subtareas
INSERT INTO Subtareas (subtareaId, tareaId, orden) VALUES
    (4, 3, 1),
    (5, 3, 2)
;

-- Insertar datos de ejemplo en la tabla Empleados
INSERT INTO Empleados (empleadoId, dni, nombre) VALUES
    (1, '12345678A', 'Juan Pérez'),
    (2, '87654321B', 'María López'),
    (3, '11223344C', 'Carlos García'),
    (4, '44332211D', 'Ana Rueda')
;

-- Insertar datos de ejemplo en la tabla PeriodosCargos
INSERT INTO PeriodosCargos (periodoCargoId, empleadoId, rolId, fInicio, fFin)
VALUES
    (1, 1, 1, '2023-12-01', null),
    (2, 2, 2, '2024-01-01', null),
    (3, 3, 3, '2024-01-01', null),
    (4, 4, 3, '2024-01-01', null)
;

-- Insertar datos de ejemplo en la tabla PeriodosCargos
INSERT INTO PeriodosTareas (periodoTareaId, empleadoId, tareaId, fInicio, fFin)
VALUES
    (1, 1, 1, '2024-01-01', '2024-01-30'),
    (2, 2, 2, '2024-02-01', '2024-03-15'),
    (3, 3, 3, '2024-03-01', '2024-04-30'),
    (4, 4, 4, '2024-05-01', '2024-06-15')
;
```

11. Consultas

Proyectos/queries.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2024
-- Descripción: Script con las consultas
--

-- Empleados con roles en proyectos:
SELECT e.nombre, r.nombre rol, pc.fInicio, pc.fFin
FROM Empleados e JOIN PeriodosCargos pc
  ON e.empleadoId = pc.empleadoId JOIN Roles r
  ON r.rolId  = pc.rolId
;

-- Empleados con tareas en proyectos:
SELECT e.nombre, t.descripcion, pt.fInicio, pt.fFin
FROM Empleados e JOIN PeriodosTareas pt
  ON e.empleadoId = pt.empleadoId JOIN Tareas t
  ON t.tareaId = pt.tareaId
;
```

```sql
-- Empleados que trabajan en proyectos (roles o tareas):
SELECT e.nombre, r.nombre rol, pc.fInicio, pc.fFin
FROM Empleados e JOIN PeriodosCargos pc
  ON e.empleadoId = pc.empleadoId JOIN Roles r
  ON r.rolId  = pc.rolId
UNION
SELECT e.nombre, t.descripcion, pt.fInicio, pt.fFin
FROM Empleados e JOIN PeriodosTareas pt
  ON e.empleadoId = pt.empleadoId JOIN Tareas t
  ON t.tareaId = pt.tareaId
;

-- Empleados que trabajan en proyectos (roles y tareas):
SELECT e.nombre, r.nombre rol, t.descripcion
FROM Empleados e JOIN PeriodosCargos pc
  ON e.empleadoId = pc.empleadoId JOIN Roles r
  ON r.rolId  = pc.rolId JOIN PeriodosTareas pt
  ON e.empleadoId = pt.empleadoId JOIN Tareas t
  ON t.tareaId = pt.tareaId
;

-- Tareas asignadas a empleados en el proyecto 1:
SELECT e.nombre, t.descripcion, pt.fInicio, pt.fFin
FROM Empleados e JOIN PeriodosTareas pt
  ON e.empleadoId = pt.empleadoId JOIN Tareas t
  ON t.tareaId = pt.tareaId
WHERE t.proyectoId = 1
;

-- Número de tareas asignadas a los empleados del proyecto 1:
SELECT e.empleadoId, e.nombre, count(*)
FROM Empleados e JOIN PeriodosTareas pt
  ON e.empleadoId = pt.empleadoId JOIN Tareas t
  ON t.tareaId = pt.tareaId
WHERE t.proyectoId = 1
GROUP BY e.empleadoId
;

-- Listado de Roles asignados a los proyectos:
SELECT e.empleadoId, e.nombre, r.nombre rol, pc.fInicio, pc.fFin
FROM Empleados e JOIN PeriodosCargos pc
  ON e.empleadoId = pc.empleadoId JOIN Roles r
  ON r.rolId  = pc.rolId
GROUP BY e.empleadoId
;

-- Listado de tareas de los proyectos:
SELECT e.empleadoId, e.nombre, t.descripcion, pt.fInicio, pt.fFin
FROM Empleados e JOIN PeriodosTareas pt
  ON e.empleadoId = pt.empleadoId JOIN Tareas t
  ON t.tareaId = pt.tareaId
GROUP BY e.empleadoId
;

-- Número de subtareas de cada tareas de los proyectos:
SELECT t.tareaId, COUNT(*)
```

```sql
FROM Tareas t LEFT JOIN Subtareas s
  ON t.tareaId = s.tareaId
GROUP BY t.tareaId
;

-- Empleados con subtareas en proyectos:
SELECT e.nombre, t.descripcion, pt.fInicio, pt.fFin
FROM Empleados e JOIN PeriodosTareas pt
  ON e.empleadoId = pt.empleadoId JOIN Tareas t
  ON t.tareaId = pt.tareaId JOIN Subtareas s
  ON t.tareaId = s.tareaId
;
```